What are confidence intervals and why do we need them?

In this video, you will learn

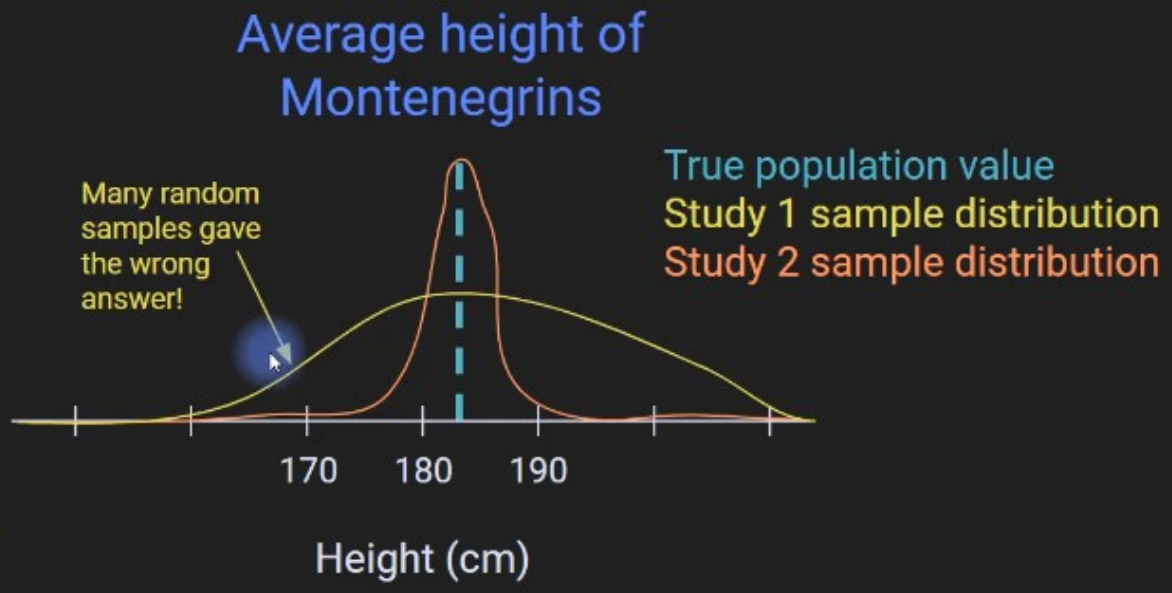
- What “confidence intervals” are.
- The factors that influence our statistical confidence.
- Why confidence intervals are important.

Why confidence intervals are necessary

Experiment methods:

Study 1: Measure height in 10 randomly selected people, compute average. Repeat 500 times.

Study 2: Measure height in 80 randomly selected people, compute average. Repeat 500 times.

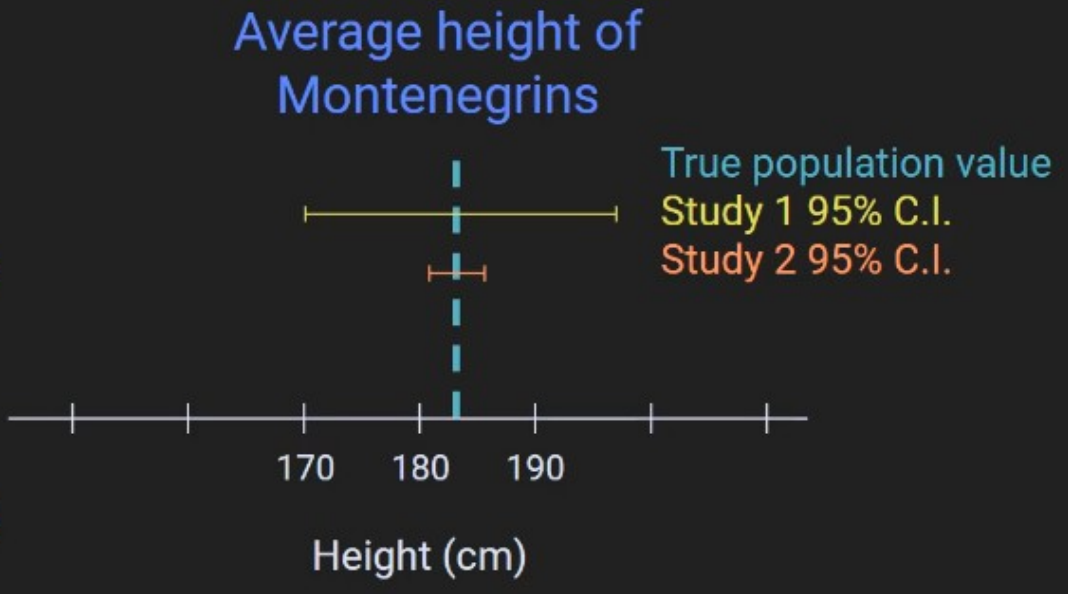


Why confidence intervals are necessary

Experiment methods:

Study 1: Measure height in 10 randomly selected people, compute average. Repeat 500 times.

Study 2: Measure height in 80 randomly selected people, compute average. Repeat 500 times.



Confidence interval: definition

Confidence interval:
The probability that an unknown population parameter falls within a range of values in repeated samples*.

$$P(L < \mu < U) = c$$

Typical confidence interval probabilities:
95%, 99%, or 90%

*The proportion of a large number of samples that will include the population parameter within its confidence interval.

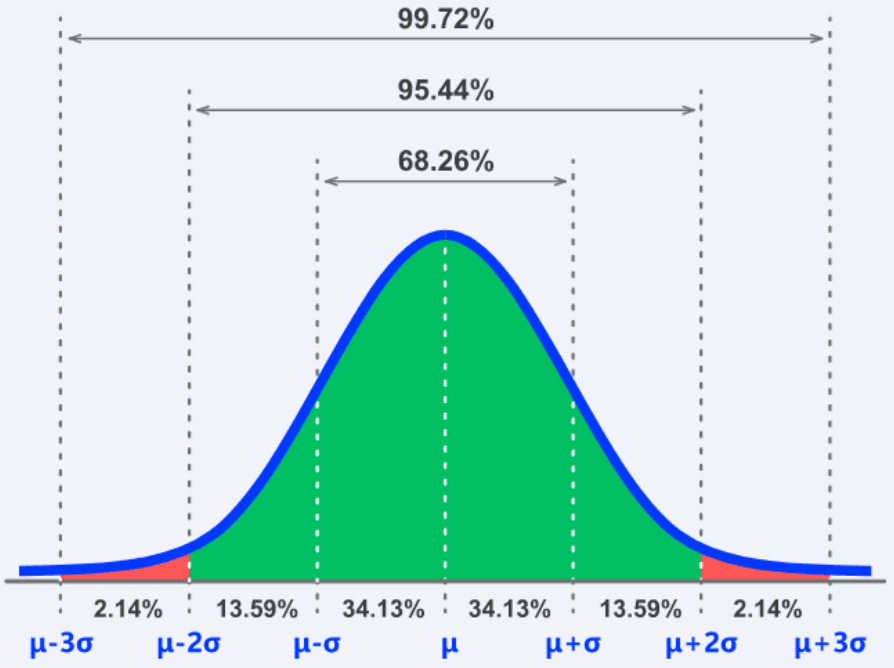
Factors that influence confidence intervals

Confidence intervals are influenced by the sample size and variance.

When sample size is larger, confidence intervals are closer together.

When the variance is smaller, confidence intervals are closer together.

Confidence Interval



appinio

Importance of Confidence Interval in Statistical Analysis

Quantifying Uncertainty: Confidence intervals provide a measure of uncertainty around point estimates, allowing researchers to assess the reliability and precision of their findings. Confidence intervals help avoid overconfidence in statistical estimates by acknowledging and quantifying uncertainty.

Inference and Decision-making: Confidence intervals play a crucial role in statistical inference and decision-making. They enable researchers to make inferences about population parameters based on sample data, guiding decisions in research, business, healthcare, and policy-making.

Comparing Groups or Treatments: Confidence intervals facilitate comparisons between groups or treatments by providing a range of plausible values for population parameters. Whether comparing means, proportions, or other statistics, confidence intervals help assess the significance and magnitude of differences.

Sample Size Determination: Confidence intervals inform sample size determination for research studies. By specifying the desired level of precision and confidence, researchers can calculate the sample size needed to achieve their study objectives while minimizing costs and resources.

Communicating Results: Confidence intervals offer a concise way to communicate the precision and uncertainty of statistical estimates to stakeholders, including researchers, policymakers, and the general public. They provide a clear indication of the range within which the true population parameter is likely to fall.

Robustness to Assumptions: Unlike point estimates, which can be sensitive to outliers or violations of distributional assumptions, confidence intervals are more robust and provide a more comprehensive picture of the underlying uncertainty. They offer a flexible approach to statistical analysis, particularly in situations where parametric assumptions may not hold.

Quality Control and Process Improvement: In quality control and process improvement, confidence intervals are used to monitor and assess the performance of systems and processes. By tracking confidence intervals over time, organizations can identify trends, detect deviations from expected performance, and implement corrective actions as needed.

Scientific Reproducibility: Confidence intervals contribute to the transparency and reproducibility of scientific research by quantifying the uncertainty inherent in statistical estimates. Replication studies can use confidence intervals to assess the consistency and generalizability of findings across different samples or settings.

Decision-making under Uncertainty: In decision-making contexts, confidence intervals provide decision-makers with a framework for considering uncertainty and variability in their choices. Whether evaluating the effectiveness of interventions, assessing risks, or allocating resources, confidence intervals inform more informed and robust decisions.

Computing confidence intervals via formula

In this video, you will learn

- The “analytic” method for computing confidence intervals.
- The assumption for applying the formula.
- Why variance and sample size influence confidence intervals.

The formula for C.I.

$$\text{C.I.} = \overline{x} \pm t^*(k)\frac{s}{\sqrt{n}}$$

x: sample mean
t^*: t-value with k degrees of freedom
s: sample standard deviation
n: sample size

The formula for C.I.

$$\text{C.I.} = \overline{x} \pm \frac{s}{\sqrt{n}}$$

s gets smaller, C.I. gets smaller.
n gets larger, C.I. gets smaller.

The formula for C.I.

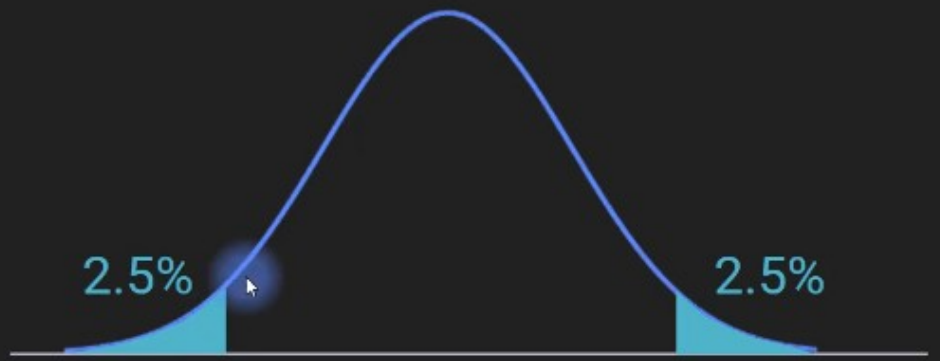
$$\text{C.I.} = \overline{x} \pm t^*(k)\frac{s}{\sqrt{n}}$$

t^*: t-value associated with *one tail* of the confidence interval: *(1-C)/2 = tinv((1-.95)/2,n-1)*

example: 95% C.I., n=20 ⇒ t^* = 2.093

The formula for C.I.

$$\text{C.I.} = \overline{x} \pm t^*(k)\frac{s}{\sqrt{n}}$$

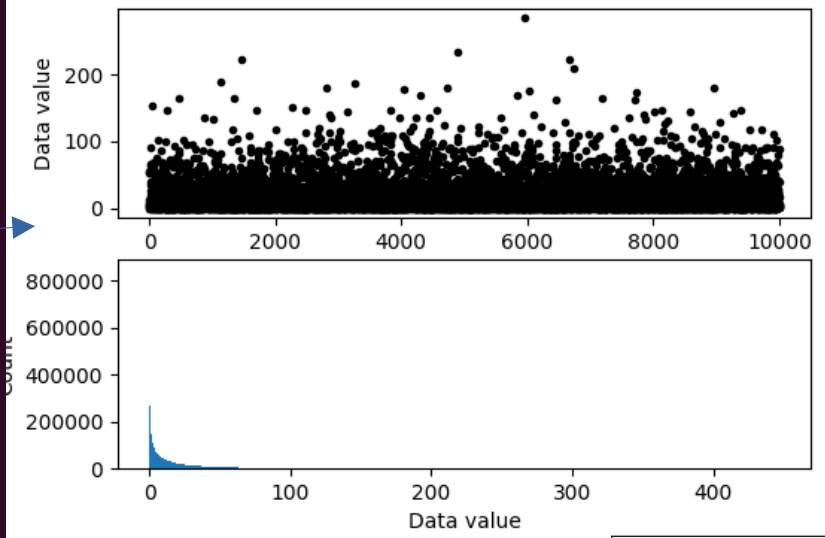


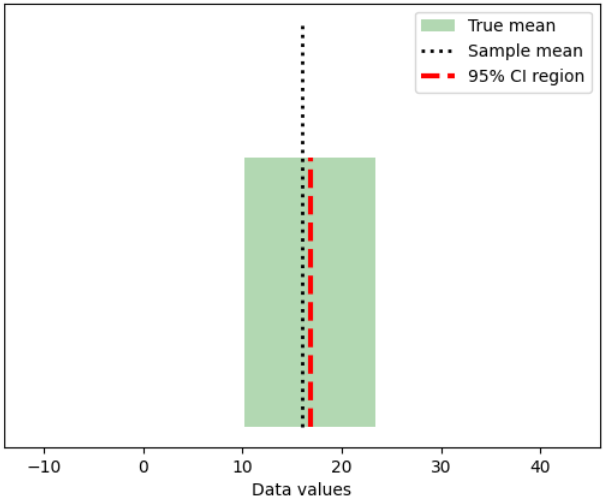
Assumptions underlying the C.I. formula

$$\text{C.I.} = \overline{x} \pm t^*(k)\frac{s}{\sqrt{n}}$$

Key assumption:
s is an appropriate measure of variability

```python
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats
from matplotlib.patches import Polygon
## simulate data
popN = int(1e7)  # lots and LOTS of data!!
# the data (note: non-normal!)
population = (4*np.random.randn(popN))**2
# we can calculate the exact population mean
popMean = np.mean(population)
# let's see it
fig,ax = plt.subplots(2,1,figsize=(6,4))
# only plot every 1000th sample
ax[0].plot(population[::1000],'k.')
ax[0].set_xlabel('Data index')
ax[0].set_ylabel('Data value')
ax[1].hist(population,bins='fd')
ax[1].set_ylabel('Count')
ax[1].set_xlabel('Data value')
plt.show()
## draw a random sample
# parameters
samplesize = 40
confidence = 95 # in percent
# compute sample mean
randSamples = np.random.randint(0,popN,samplesize)
samplemean  = np.mean(population[randSamples])
samplestd   = np.std(population[randSamples],ddof=1)
# compute confidence intervals
citmp = (1-confidence/100)/2
confint = samplemean + stats.t.ppf([citmp, 1-citmp],samplesize-1) * samplestd/np.sqrt(samplesize)
# graph everything
fig,ax = plt.subplots(1,1)
y = np.array([ [confint[0],0],[confint[1],0],[confint[1],1],[confint[0],1] ])
p = Polygon(y,facecolor='g',alpha=.3)
ax.add_patch(p)
# now add the lines
ax.plot([popMean,popMean],[0, 1.5],'k:',linewidth=2)
ax.plot([samplemean,samplemean],[0, 1],'r--',linewidth=3)
ax.set_xlim([popMean-30, popMean+30])
ax.set_yticks([])
ax.set_xlabel('Data values')
ax.legend(('True mean','Sample mean','%g%% CI region'%confidence))
plt.show()
## repeat for large number of samples
# parameters
samplesize = 50
confidence = 95  # in percent
numExperiments = 5000
withinCI = np.zeros(numExperiments)
# part of the CI computation can be done outside the loop
citmp = (1-confidence/100)/2
CI_T  = stats.t.ppf([citmp, 1-citmp],samplesize-1)
sqrtN = np.sqrt(samplesize)
for expi in range(numExperiments):
    # compute sample mean and CI as above
    randSamples = np.random.randint(0,popN,samplesize)
    samplemean  = np.mean(population[randSamples])
    samplestd   = np.std(population[randSamples],ddof=1)
    confint     = samplemean + CI_T * samplestd/sqrtN
    # determine whether the True mean is inside this CI
    if popMean>confint[0] and popMean<confint[1]:
        withinCI[expi] = 1
print('%g%% of sample C.I.''s contained the true population mean.'%(100*np.mean(withinCI)))
```






```
warnings.warn( Unable to import Axes3D. This may be due
91.7% of sample C.I.s contained the true population mean.
```

Confidence intervals via bootstrapping

In this video, you will learn

- The nonparametric method for computing confidence intervals via bootstrapping (a.k.a. resampling).
- The advantages and disadvantages of parametric vs. nonparametric confidence intervals.

Bootstrapping C.I.: The idea

Instead of using a formula to compute confidence intervals, compute them directly based on the data.

This is done by repeatedly randomly resampling from your dataset.

Thus, pretend that your sample is the population, and the resampling is the sample.

Bootstrapping: mechanism

Original dataset $x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9, x_{10}$ $\bar{x} = y_o$

Resample with replacement

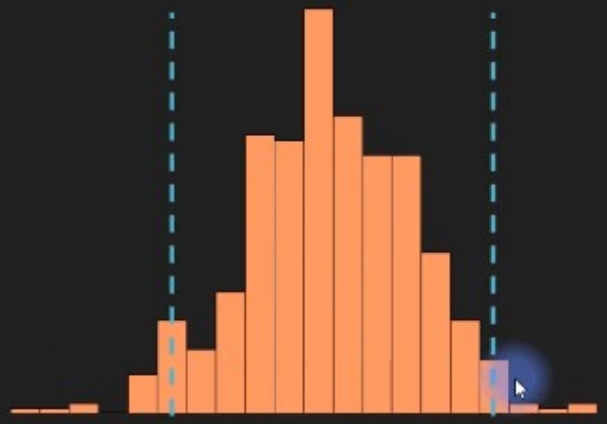
Bootstrapped dataset 1 $x_6, x_6, x_1, x_8, x_9, x_7, x_8, x_{10}, x_1, x_2$ $\bar{x} = y_1$

Bootstrapped dataset 2 $x_8, x_7, x_3, x_3, x_{10}, x_1, x_8, x_4, x_9, x_5$ $\bar{x} = y_2$

⋮

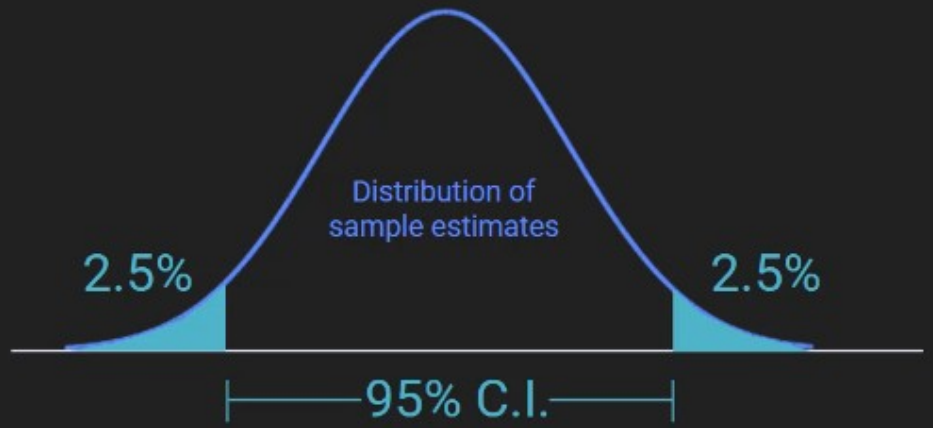
500 times

$y_1, y_2, \ldots, y_{500}$

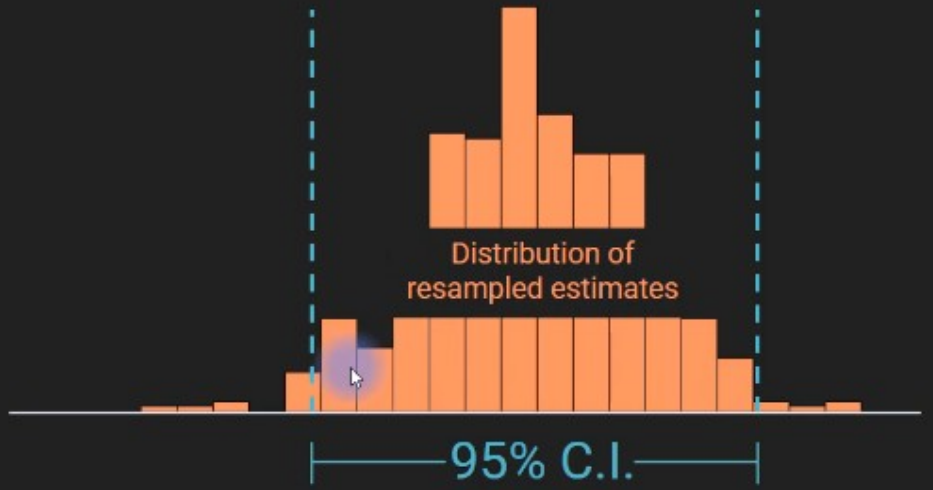


Parametric vs. nonparametric C.I.

Parametric:
Based on formula
and assumptions



Nonparametric:
Based on algorithm
and data



Bootstrapping: pros and cons

Advantages:

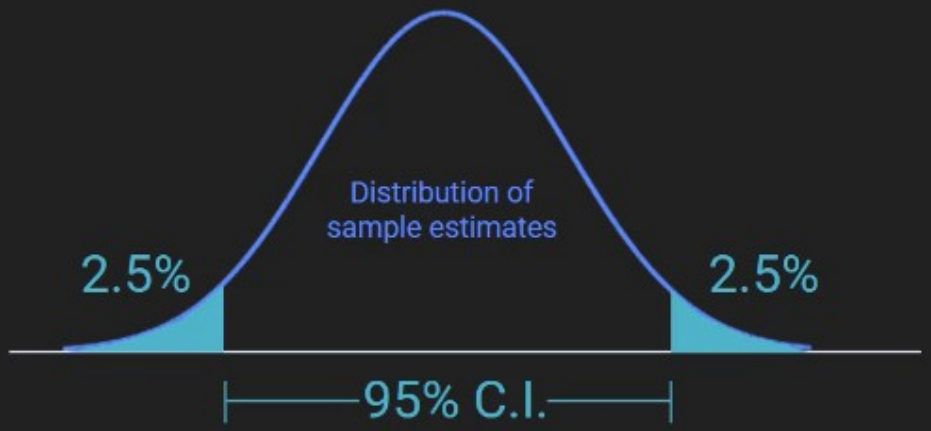
- Works for any kind of parameter (mean, variance, correlation, median, etc.).
- Useful for limited data (e.g., no experiment repetitions).
- Not based on assumptions of normality.

Limitations:

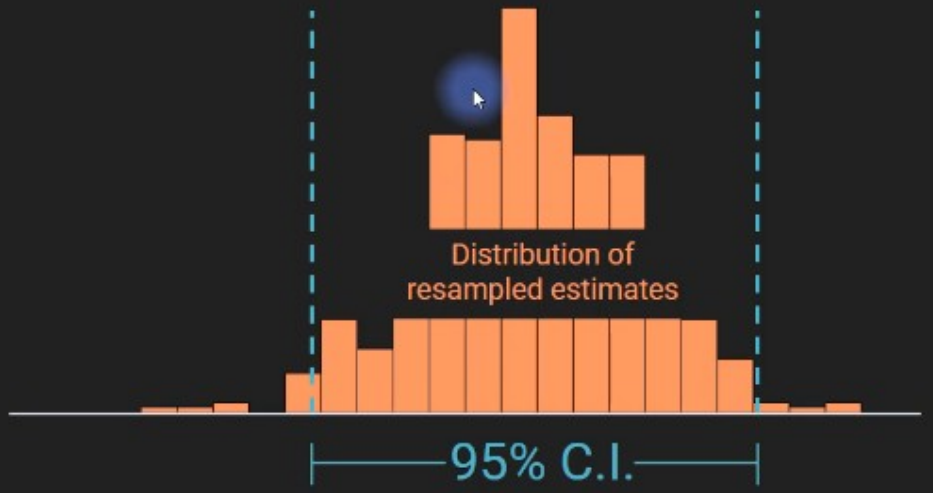
- Gives (slightly) different results each time.
- Can be time-consuming for large datasets.
- Sample must be a good representation of the population.

Parametric vs. nonparametric C.I.

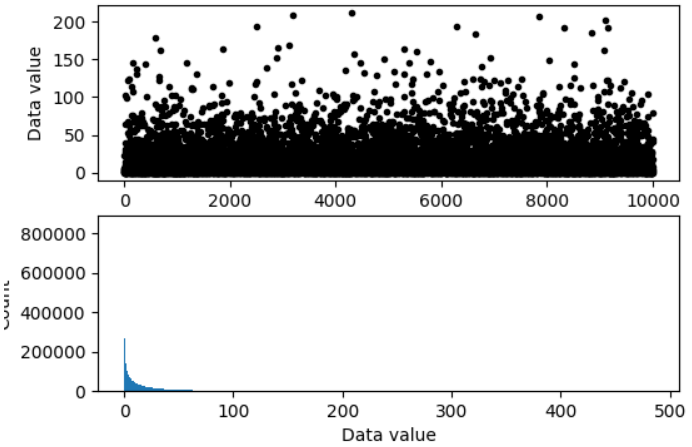
Parametric:
Based on formula
and assumptions

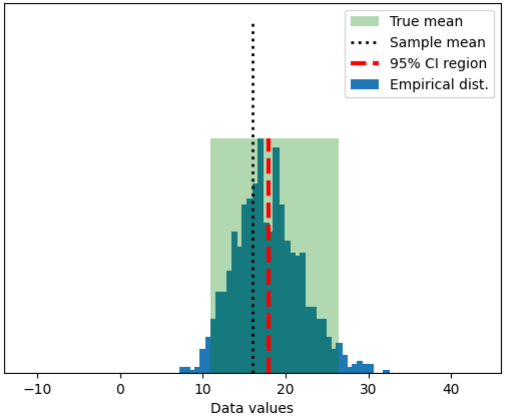


Nonparametric:
Based on algorithm
and data



```python
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats
from matplotlib.patches import Polygon
## simulate data
popN = int(1e7)  # lots and LOTS of data!!
# the data (note: non-normal!)
population = (4*np.random.randn(popN))**2
# we can calculate the exact population mean
popMean = np.mean(population)
# let's see it
fig,ax = plt.subplots(2,1,figsize=(6,4))
# only plot every 1000th sample
ax[0].plot(population[::1000],'k.')
ax[0].set_xlabel('Data index')
ax[0].set_ylabel('Data value')
ax[1].hist(population,bins='fd')
ax[1].set_ylabel('Count')
ax[1].set_xlabel('Data value')
plt.show()
## draw a random sample
# parameters
samplesize = 40
confidence = 95 # in percent
# compute sample mean
randSamples = np.random.randint(0,popN,samplesize)
sampledata  = population[randSamples]
samplemean  = np.mean(population[randSamples])
samplestd   = np.std(population[randSamples]) # used later for analytic solution
### now for bootstrapping
numBoots  = 1000
bootmeans = np.zeros(numBoots)
# resample with replacement
for booti in range(numBoots):
    bootmeans[booti] = np.mean( np.random.choice(sampledata,samplesize) )
# find confidence intervals
confint = [0,0] # initialize
confint[0] = np.percentile(bootmeans,(100-confidence)/2)
confint[1] = np.percentile(bootmeans,100-(100-confidence)/2)
## graph everything
fig,ax = plt.subplots(1,1)
# start with histogram of resampled means
y,x = np.histogram(bootmeans,40)
y = y/max(y)
x = (x[:-1]+x[1:])/2
ax.bar(x,y)
y = np.array([ [confint[0],0],[confint[1],0],[confint[1],1],[confint[0],1] ])
p = Polygon(y,facecolor='g',alpha=.3)
ax.add_patch(p)
# now add the lines
ax.plot([popMean,popMean],[0, 1.5],'k:',linewidth=2)
ax.plot([samplemean,samplemean],[0, 1],'r--',linewidth=3)
ax.set_xlim([popMean-30, popMean+30])
ax.set_yticks([])
ax.set_xlabel('Data values')
ax.legend(('True mean','Sample mean','%g%% CI region'%confidence,'Empirical dist.'))
plt.show()
## compare against the analytic confidence interval
# compute confidence intervals
citmp = (1-confidence/100)/2
confint2 = samplemean + stats.t.ppf([citmp, 1-citmp],samplesize-1) * samplestd/np.sqrt(samplesize)
print('Empirical: %g - %g'%(confint[0],confint[1]))
print('Analytic:  %g - %g'%(confint2[0],confint2[1]))
```






```
Empirical: 10.9547 - 26.4124
Analytic:  9.62166 - 26.3085
```

Misconceptions about confidence intervals

In this video, you will learn

- The precise interpretation of confidence intervals.
- Misinterpretations to avoid.

Misinterpretation #1

$$P(L < \mu < U) = \overline{x} \pm t^*(k)\frac{s}{\sqrt{n}}$$

Incorrect:
"I am 95% confident that the population mean is the sample mean."

Correct:
"95% of confidence intervals in repeated samples will contain the true population mean."

Misinterpretation #2

$$P(L < \mu < U) = \overline{x} \pm t^*(k)\frac{s}{\sqrt{n}}$$

Incorrect:
"I am 95% confident that the population mean is within the C.I. in my dataset."

Correct:
See previous slide. Also, the confidence refers to the estimate, not to the the population parameter.

Misinterpretation #3

$$P(L < \mu < U) = \overline{x} \pm t^*(k)\frac{s}{\sqrt{n}}$$

Incorrect:
"95% of the data are between *L* and *U*."

Reason:
The C.I. is not based on the raw data; it's based on the descriptive statistics of the sample data.

Misinterpretation #4

$$P(L < \mu < U) = \overline{x} \pm t^*(k)\frac{s}{\sqrt{n}}$$

Incorrect:
"Confidence intervals for two parameters overlap; therefore, they cannot be significantly different."

Reason:
The C.I. refers to the estimate of a parameter, not to the relationship between parameters.